Unit Commitment

This examples covers Unit Commitment, a classical operations research problem that arises in the operation of electrical networks. In this problem, multiple power generation units with different characteristics are dispatched to meet expected electricity demand. A unit can be on or off, with a startup cost associated with transitioning from off to on, and power output that must lie in a specified range while the unit is on. The model is specified over discrete time periods, and decides which units to turn on, and when, in order to satisfy demand for each time period. The model also captures a reserve requirement, where the selected power plants must be capable of increasing their output, while still respecting their maximum output, in order to cope with the situation where actual demand exceeds predicted demand.

This model is based on example 15 from the fifth edition of Model Building in Mathematical Programming, by H. Paul Williams on pages 270-271 and 325-326, and is adapted from an existing Gurobi notebook here which uses Python data structures to build the model.

In [1]:
```
import pandas as pd
import gurobipy as gp
from gurobipy import GRB
import gurobipy_pandas as gppd

gppd.set_interactive()
```

In [2]:
```
# Hidden cell to avoid licensing messages
# when docs are generated.
with gp.Model():
    pass
```

```
Set parameter WLSAccessID
Set parameter WLSSecret
Set parameter LicenseID to value 2723234
Academic license 2723234 - for non-commercial use only - registered to nz___@frsf.utn.edu.ar
```

Data Schema

Each generator has properties which remain fixed over all time periods:

- `num_available` : number of available generating units
- `min_output` : minimum generation in MWh for each active generator
- `max_output` : maximum generation in MWh for each active generator
- `cost_per_hour` : cost per hour per active generator
- `marginal cost` : cost per MWh for generation above min_output
- `startup_cost` : fixed cost incurred for starting a generator in an interval
- `state0` : number of generators active before the first period

Input data for the generators is stored in a DataFrame with the generator class name as the index.

In [3]:
```
# Load and check the generator data
generator_data = pd.read_csv(
    "data/generators.csv",
    index_col="generator_class",
)
generator_data
```

Out[3]:

generator_class	num_available	min_output	max_output	cost_per_hour	marginal_cost	startup_cost	state0
thermal1	12	850.0	2000.0	1000.0	2.0	2000.0	0
thermal2	10	1250.0	1750.0	2600.0	1.3	1000.0	0
thermal3	5	1500.0	4000.0	3000.0	3.0	500.0	0

Each time period has the following data:

- `expected_demand` : predicted MWh demand, which the solution will meet exactly
- `minimum_capacity` : value in MWh above the predicted demand; the total online generation capacity must exceed this value

Input data for the time periods is stored in a DataFrame with the time periods as the index.

In [4]:
```
# Load and check the time period data
time_period_data = pd.read_csv(
    "data/time_periods.csv",
    parse_dates=["time_period"],
    index_col="time_period",
)
time_period_data
```

Out[4]:

time_period	expected_demand	minimum_active_capacity
2024-07-19 06:00:00	15000.0	17250.0
2024-07-19 07:00:00	30000.0	34500.0
2024-07-19 08:00:00	25000.0	28750.0
2024-07-19 09:00:00	40000.0	46000.0
2024-07-19 10:00:00	27000.0	31050.0

Create the model

In [5]:
```
model = gp.Model()
```

Add Time-Expanded Variables

The model has three variable types capturing the state of each generator class:

- `output` : The total output of all generators in the class in the given time period (continuous)
- `num_active` : The number of active generators of the class in the given time period (integer, upper bounded by number of available generators)
- `num_startup` : The number of active generators of the class which start up in the given time period (integer)

One variable of each type is needed for every generator class and time period. To create this 'time-expanded' formulation we need to take the product of the two indexes in our input data. This is done using pandas' MultiIndex.from_product method.

Using this time-expanded index, we'll then use the DataFrame accessors from gurobipy-pandas to create our variables.

In [6]:
```
# Simplifies variable names
short_time = {"time_period": lambda index: index.strftime("%H%M")}

# Construct time-expanded index and add variables
generators = (
    # Create a new dataframe for the time-expanded index
    pd.DataFrame(
        index=pd.MultiIndex.from_product([generator_data.index, time_period_data.index])
    )
    .join(generator_data)
    # Create continuous variables (one per row) for generator output
    .gppd.add_vars(model, name="output", index_formatter=short_time)
    # Create integer variables for the number of active generators
    .gppd.add_vars(
        model,
        vtype=GRB.INTEGER,
        ub="num_available",  # Use num_available from the input data as a bound
        name="num_active",
        index_formatter=short_time,
    )
    # Create non-negative integer variables capturing generator startups
    .gppd.add_vars(
        model, vtype=GRB.INTEGER, name="num_startup", index_formatter=short_time
    )
)
```

The resulting `generators` DataFrame will be used to create constraints. Note that it contains both data columns and Gurobi variables as columns. This allows us to use standard pandas operations to build constraint expressions.

In [7]:
```
generators
```

Out[7]:

generator_class	time_period	num_available	min_output	max_output	cost_per_hour	marginal_cost	startup_cost	state0	o
thermal1	2024-07-19 06:00:00	12	850.0	2000.0	1000.0	2.0	2000.0	0	<gurc output[thermal1,0
	2024-07-19 07:00:00	12	850.0	2000.0	1000.0	2.0	2000.0	0	<gurc output[thermal1,0
	2024-07-19 08:00:00	12	850.0	2000.0	1000.0	2.0	2000.0	0	<gurc output[thermal1,0
	2024-07-19 09:00:00	12	850.0	2000.0	1000.0	2.0	2000.0	0	<gurc output[thermal1,0
	2024-07-19 10:00:00	12	850.0	2000.0	1000.0	2.0	2000.0	0	<gurc output[thermal1,1
thermal2	2024-07-19 06:00:00	10	1250.0	1750.0	2600.0	1.3	1000.0	0	<gurc output[thermal2,0
	2024-07-19 07:00:00	10	1250.0	1750.0	2600.0	1.3	1000.0	0	<gurc output[thermal2,0
	2024-07-19 08:00:00	10	1250.0	1750.0	2600.0	1.3	1000.0	0	<gurc output[thermal2,0
	2024-07-19 09:00:00	10	1250.0	1750.0	2600.0	1.3	1000.0	0	<gurc output[thermal2,0
	2024-07-19 10:00:00	10	1250.0	1750.0	2600.0	1.3	1000.0	0	<gurc output[thermal2,1
thermal3	2024-07-19 06:00:00	5	1500.0	4000.0	3000.0	3.0	500.0	0	<gurc output[thermal3,0
	2024-07-19 07:00:00	5	1500.0	4000.0	3000.0	3.0	500.0	0	<gurc output[thermal3,0
	2024-07-19 08:00:00	5	1500.0	4000.0	3000.0	3.0	500.0	0	<gurc output[thermal3,0
	2024-07-19 09:00:00	5	1500.0	4000.0	3000.0	3.0	500.0	0	<gurc output[thermal3,0
	2024-07-19 10:00:00	5	1500.0	4000.0	3000.0	3.0	500.0	0	<gurc output[thermal3,1

Demand Constraints

There are two types of demand constraints:

1. The total output of all generators in each time period must match the expected demand
2. The active generators in each time period must be able to meet the reserve demand

In [8]:
```
# Constrain that predicted demand is exactly satisfied
demand_constraint = gppd.add_constrs(
    model,
    generators.groupby("time_period")["output"].sum(),
    GRB.EQUAL,
    time_period_data["expected_demand"],
    index_formatter=short_time,
)
```

In [9]:
```
# Constrain that the active generators during each time
# period are capable of meeting the reserve demand.
active_capacity = (
    (generators["max_output"] * generators["num_active"])
    .groupby("time_period").sum()
).rename("active_capacity")
active_capacity_constraint = gppd.add_constrs(
    model,
    active_capacity,
    GRB.GREATER_EQUAL,
    time_period_data["minimum_active_capacity"],
    index_formatter=short_time,
)
```

Note that we keep total online capacity as a series of expressions. This way we can directly use it in analysis of the results.

In [10]:
```
active_capacity.to_frame()
```

Out[10]:

time_period	active_capacity
2024-07-19 06:00:00	2000.0 num_active[thermal1,0600] + 1750.0 num_...
2024-07-19 07:00:00	2000.0 num_active[thermal1,0700] + 1750.0 num_...
2024-07-19 08:00:00	2000.0 num_active[thermal1,0800] + 1750.0 num_...
2024-07-19 09:00:00	2000.0 num_active[thermal1,0900] + 1750.0 num_...
2024-07-19 10:00:00	2000.0 num_active[thermal1,1000] + 1750.0 num_...

Output Constraints

Each generator class is constrained within it's operating limits.

In [11]:
```
df = (
    generators
    .gppd.add_constrs(
        model,
        "output >= min_output * num_active",
        name="lower_limit",
        index_formatter=short_time,
    )
    .gppd.add_constrs(
        model,
        "output <= max_output * num_active",
        name="constr_max_output",
        index_formatter=short_time,
    )
)
```

Startup Constraints

The startup variables will be used to capture the cost associated with starting a generator during a time period. For this we need a rolling-window constraint such that startups capture the difference between the number of generators online in adjacent time periods.

In [12]:
```
# Constrain the relationship between active generators and startups.

def startup_constraints(group):
    group = group.sort_index()
    return gppd.add_constrs(
        model,
        group["num_startup"].iloc[1:],
        GRB.GREATER_EQUAL,
        group["num_active"].diff().dropna(),
        name="startup",
        index_formatter=short_time,
    )

startup = generators.groupby("generator_class").apply(startup_constraints).droplevel(0)
```

This groupby + diff operation creates constraints of this form (these can be inspected by writing the model to an LP file using `model.write("unit-commitment.lp")`):

```
    startup[thermal1,0900]: num_active[thermal1,0800]
       - num_active[thermal1,0900] + num_startup[thermal1,0900] >= 0
```

i.e. the number of generators started at 9am, is at least as large as difference between the number of active generators at 9am and the number of active generators at 8am. `num_startup` is a non-negative integer, and it has a cost penalty associated with it in the objective function, so we can be sure it will capture the number of startups correctly.

In [13]:
```
# Separately capture the startups at time period 0.

time_period_1 = generators.sort_index().groupby("generator_class").first()
initial_startup = gppd.add_constrs(
    model,
    time_period_1["num_startup"],
    GRB.GREATER_EQUAL,
    time_period_1["num_active"] - generator_data["state0"],
    name="initial_startup",
    index_formatter=short_time,
)
```

Objective Function

The total cost objective is now easy to compute:

In [14]:
```python
# Minimize total cost objective
model.setObjective(
    (
        # Fixed hourly costs for started generators
        generators["cost_per_hour"] * generators["num_active"]
        # Marginal hourly cost of additional generation above the minimum
        + generators["marginal_cost"]
        * (generators["output"] - generators["num_active"] * generators["min_output"])
        # Startup costs for newly active generators in each time period
        + generators["startup_cost"] * generators["num_startup"]
    ).sum(),
    sense=GRB.MINIMIZE
)
```

Solve the model

In [15]:
```python
model.optimize()
```

```
Gurobi Optimizer version 13.0.0 build v13.0.0rc1 (win64 - Windows 11+.0 (26200.2))

CPU model: AMD Ryzen 5 7430U with Radeon Graphics, instruction set [SSE2|AVX|AVX2]
Thread count: 6 physical cores, 12 logical processors, using up to 12 threads

Academic license 2723234 - for non-commercial use only - registered to nz___@frsf.utn.edu.ar
Optimize a model with 55 rows, 45 columns and 132 nonzeros (Min)
Model fingerprint: 0xf6611d63
Model has 45 linear objective coefficients
Variable types: 15 continuous, 30 integer (0 binary)
Coefficient statistics:
  Matrix range     [1e+00, 4e+03]
  Objective range  [1e+00, 2e+03]
  Bounds range     [5e+00, 1e+01]
  RHS range        [2e+04, 5e+04]
Found heuristic solution: objective 327950.00000
Presolve removed 12 rows and 9 columns
Presolve time: 0.04s
Presolved: 43 rows, 36 columns, 113 nonzeros
Found heuristic solution: objective 319750.00000
Variable types: 8 continuous, 28 integer (0 binary)

Root relaxation: objective 2.563143e+05, 26 iterations, 0.01 seconds (0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0 256314.286    0    5 319750.000 256314.286  19.8%     -    0s
H    0     0                    257055.00000 256314.286  0.29%     -    0s
H    0     0                    256565.00000 256314.286  0.10%     -    0s
H    0     0                    256500.00000 256314.286  0.07%     -    0s

Cutting planes:
  Gomory: 1
  MIR: 2

Explored 1 nodes (26 simplex iterations) in 0.11 seconds (0.00 work units)
Thread count was 12 (of 12 available processors)

Solution count 5: 256500 256565 257055 ... 327950

Optimal solution found (tolerance 1.00e-04)
Best objective 2.565000000000e+05, best bound 2.565000000000e+05, gap 0.0000%
```

Extract results

Results are extracted using Series accessors. Note that after extracting the results, we can `close()` the model and proceed to analyse results using only pandas operations.

In [16]:
```python
# Extract all variable values
solution = pd.DataFrame(
    dict(
        output=generators["output"].gppd.X,
        num_active=generators["num_active"].gppd.X,
        num_startup=generators["num_startup"].gppd.X,
    )
)

# Extract some additional results for comparison
results = pd.DataFrame(
    {
        "Demand": time_period_data["expected_demand"],
        "Min. Active Capacity": time_period_data["minimum_active_capacity"],
        "Active Capacity": active_capacity.gppd.get_value(),
        "Excess": (-active_capacity_constraint.gppd.Slack),
```

```
    }
)

# After extracting all results, close the model.
model.close()
```

Analysis

Briefly; show the solution meeting the reserve demand and the excess capacity online.

In [17]:
```
%matplotlib inline
%config InlineBackend.figure_formats = ['svg']

import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_palette(sns.color_palette("deep"))

plt.figure(figsize=(8, 4))
results.plot.line(ax=plt.gca())
plt.xlabel("Time")
plt.ylabel("MWh")
plt.ylim([0, 50000])
plt.legend(loc=2);
```